Ensemble Session Inspection Agent

> *The AI-powered first responder for Ensemble productions — session diagnostics in 30 seconds, not 30 minutes.*

Executive Summary

Every night, somewhere on call, an Ensemble integration engineer stares at a failed session with six tabs open — the Visual Trace, the Event Log, the Rule Log, the Business Process source code, and two more they'd rather not have needed. Twenty minutes of manual correlation later, they know why the message failed. The next engineer who hits the same session starts from scratch.

The **Ensemble Session Inspection Agent** replaces that process with a conversation. Ask *"What happened in session 42751?"* and receive a plain-English narrative — the full arc of the session, which components touched the message, where errors appeared, why the router chose a particular path, and what the Business Process code actually did with it. What took 20-30 minutes of expert tab-switching takes under 30 seconds.

This is not a general-purpose log-analysis tool retrofitted to IRIS. It is purpose-built for the InterSystems Ensemble session model — understanding `Ens.MessageHeader`, message correlation, Business Process logic, and the event log in the way an expert operator would — deployed as a native capability inside the Management Portal experience operators already use every day. A working diagnostic agent in under an afternoon of build time: proof that the AI Hub SDK delivers on its promise.

The Problem

An Ensemble production session creates a trail across five separate data sources: message headers, dynamically-typed message bodies, the event log, the rule log, and Business Process runtime state. The Management Portal's Visual Trace renders the choreography as a diagram — but it doesn't answer the questions operators actually need answered:

- *"Why did this message fail?"* — The error is on the response header, not the request. Easy to miss without knowing where to look.
- *"What does the body of this request actually contain?"* — The body class is arbitrary and must be looked up and deserialized at inspection time.
- *"Why did the router take this branch?"* — Requires cross-referencing a separate Rule Log table that the Visual Trace doesn't surface.
- *"What did the Business Process do between receiving this message and sending that one?"* — Requires reading ObjectScript or BPL source code and correlating it with runtime state.

Today, experienced engineers answer these questions by mentally joining five tables and reading code. Junior engineers often cannot. Both groups do it manually, every time. Boomi shipped a natural-language process assistant in May 2025 that helps developers understand *designed* flows — validation that the integration middleware space is ready for AI-assisted diagnostics. But Boomi helps you build a flow; this tells you why your flow failed at 2am. No equivalent exists for IRIS.

The Solution

The Ensemble Session Inspection Agent is a read-only AI agent built on the InterSystems AI Hub SDK (`%AI.Agent`, `%AI.ToolSet`, `%AI.Agent.Session`). It surfaces the full session narrative by correlating messages, logs, rule decisions, and business process state — and makes it available as a multi-turn conversation.

13 specialized, read-only tools power the agent:

Tool	What it answers
`GetSessionSummary`	Shape, duration, error count, root message class
`GetSessionTimeline`	Interleaved message + log + rule events, chronologically
`GetMessageHeaders`	All messages in session with decoded status and type
`GetMessageBody`	Decoded body contents (HL7, FHIR, custom persistent classes)
`GetMessageDetail`	Deep single-message view combining header + body + related log entries
`GetEventLog`	Filterable event log entries by session, message, and severity
`GetRuleLog`	Why routing rules fired as they did, including the reason and return value
`GetBusinessProcessInstance`	Runtime state of a BP instance during the session
`GetBusinessProcessSource`	The actual ObjectScript code or BPL XML of any Business Process
`ListBusinessProcessMethods`	Enumerate methods on any production class
`ExplainError`	Plain-English decoding of IRIS `%Status` values with pattern recognition
`FindRelatedSessions`	Cross-instance sessions via SuperSession for distributed productions
`FindSessionsByBody`	Find sessions by indexed body fields (e.g., patient MRN, org ID)

The agent is deployed in three phases — each a fully functional, demonstrable milestone — targeted at IRIS for Health 2026.2 with AI Hub EAP build 141:

Phase 1 — Terminal Bot (~1 hour): A single command launches an interactive REPL. Enter a namespace and session ID; begin a multi-turn diagnostic conversation with the full tool catalog available.

Phase 2 — Chat Tab in Visual Trace (~2.5 hours): A subclass of `EnsPortal.VisualTrace` adds a fourth "Chat" tab alongside Header, Body, and Contents. The agent knows which session is displayed and which message is currently selected. Operators ask questions in context; the agent responds using the Management Portal's own ZenMethod bridge for the AJAX layer. Multi-turn state persists across requests via `%AI.Agent.Session`, a built-in persistent class in the AI Hub SDK.

Phase 3 — Custom Message Viewer (~30 minutes): A subclass of `EnsPortal.MessageViewer` routes session-link clicks to the Chat-enabled Visual Trace. Operators bookmark one URL per namespace — the complete integrated workflow.

Data flow: User input and session metadata (message headers, event log text, ObjectScript source code) are sent to the configured LLM endpoint. Session body payloads are retrieved and decoded server-side; only the decoded content — not raw binary — leaves the IRIS server. In the initial deployment, this is scoped to non-PHI namespaces.

What Makes This Different

First responder architecture — safe by design. The agent is the first responder: always read-only, always available, never a risk to the production it is inspecting. Three layers of enforcement make this structural: method-level discipline (no mutation APIs called), AI Hub policy layer (`%AI.Policy.Authorization`), and IRIS RBAC with SELECT-only grants on `Ens.*` tables. This is the architecture choice that makes approval in regulated environments straightforward, earns operator trust immediately, and makes the PHI expansion path a natural unlock rather than a future rebuild.

It understands IRIS. General-purpose AIOps tools — Datadog, Dynatrace, Splunk — require OpenTelemetry instrumentation that Ensemble doesn't emit. They treat the production as a black box. This agent was built from the source: it knows where errors actually live (on response headers, not requests), how to resolve dynamically-typed message bodies, how to read BPL XML from `%Dictionary.XDataDefinition`, and how to decode `%Status` binary values into readable text. No general-purpose competitor can replicate this without rebuilding the domain knowledge from scratch.

It works inside the tool operators already use. No separate dashboard to learn, no data pipeline to maintain, no ingestion lag. Compiles in HSCUSTOM; accessible via bookmark URL in any target namespace via package mapping.

Built on the platform that was built for this. InterSystems shipped the AI Hub SDK precisely to enable native agents inside IRIS. The architecture (ToolSet, Agent, Session, Policy) maps directly to the diagnostic problem — the tool catalog is the first production use of the SDK's `<Query>` declarative tool pattern, which validates SQL at compile time and auto-generates JSON schemas for the LLM.

Who This Serves

Primary — Integration engineers and operators on IRIS/Ensemble productions. Whether debugging a failed ADT message on call at 11pm or explaining a misfired routing rule in a morning standup, the tool gives answers in seconds instead of minutes. Junior engineers independently resolve sessions that previously required escalation to senior staff.

Secondary — Engineering leads and QA teams. The agent narrates sessions as plain-English audit trails — ready-to-paste root cause descriptions for incident tickets, post-mortems, and code reviews.

Long-term — Any organization running IRIS interoperability workloads. Health systems, payers, HIEs, government agencies, and InterSystems' SI partner channel — organizations where integration reliability directly affects patient care and operational efficiency.

Success Criteria

Baseline win (Phase 1 Gate — minimum viable demo):

- Terminal bot operational: `Do ##class(SAgent.Main.Shell).Run("NAMESPACE", sessionId)` launches a conversation

- Agent correctly answers the four core diagnostic questions ("what happened," "what's in the body," "what did the BP do," "what does the error mean") using the 5 core Priority 1 tools
- Zero mutations to production data — verified by static code review before demo
- *This alone is a compelling, complete demonstration. Everything beyond this is additive.*

Hackathon aspiration (build incrementally until time runs out):

- Priority 2 tools added: routing explanation ("why did the router pick this path?")
- Priority 3 tools added: Business Process source code and runtime reasoning
- Phase 2 delivered: Chat tab visible in the Management Portal Visual Trace
- Phase 3 delivered: MessageViewer bookmark routes directly to the Chat-enabled trace

Adoption signals (post-hackathon):

- Time-to-diagnosis reduced by ≥50% versus manual tab-switching in structured user testing
- Junior engineers independently resolving sessions that previously required escalation
- Operators requesting the tool as part of standard namespace provisioning

Hackathon Build Strategy

The build is structured as a **milestone ladder** — each checkpoint produces a fully demonstrable product. The team targets all three phases but treats the Phase 1 Gate as the primary success condition. If time runs short, the demo pivots to the highest completed milestone; no partial or broken state is ever shown.

Milestone	What exists at this point	Demo-able?
M1 — Phase 1 Gate	Terminal bot answers 4 core diagnostic questions	✅ Yes — compelling standalone
M2 — P2 tools	Terminal bot adds routing & full message header analysis	✅ Yes — richer narrative
M3 — Phase 2a	Chat tab renders in Visual Trace (even with minimal tools)	✅ Yes — strong visual impact
M4 — Phase 2b	Full chat works end-to-end in the portal	✅ Yes — full Phase 2 demo
M5 — Phase 3	MessageViewer bookmark → chat in 2 clicks	✅ Yes — complete integrated workflow
M6 — P3/P4 tools	BP source reasoning, cross-instance discovery	✅ Yes — deepest diagnostic capability

Build order on hackathon day: M1 (all hands) → M5 (15 min, one developer) → M3 → M4 (parallel tracks) → M2/M6 (filling remaining time). Phase 3 (MessageViewer) ships early because it's a single method override; it sets up the full user journey even before Phase 2 is complete.

The architecture enforces this graceful degradation by design: Track A (agent + tools) and Track B (portal UI) are cleanly separated packages that compile independently. Any combination of completed tracks yields a working demo.

Scope

In scope for the hackathon build:

- All three deployment phases as the target (terminal → Visual Trace chat tab → Message Viewer handoff)
- Tool rollout in priority order: P1 core (5 tools) → P2 routing (2 tools) → P3 BP reasoning (3 tools) → P4 discovery (2 tools)
- Compiled in HSCUSTOM, available in target namespaces via package mapping and bookmark URL
- Non-PHI namespaces; blocking request-response chat (no streaming required for demo)
- HL7/FHIR VDoc bodies: segment-level output via `OutputToString()`; clinical field extraction not included

Explicitly not in scope for v1:

- PHI-bearing namespace support (LLM deployment decision to be made per environment)
- Proactive anomaly detection or alerting
- Portal navigation menu integration (bookmark-based access only)
- Cross-namespace session inspection in a single conversation turn

Vision

A working diagnostic agent in an afternoon proves the concept. The medium-term trajectory is clear: expand from reactive session inspection toward proactive anomaly surfacing — flagging unusual session patterns before an engineer needs to open a trace at all. Add HealthShare-specific narrative capability for XCPD, XCA, and XDS workflows. Expose the tool catalog via the MCP ecosystem so it's callable from Claude Desktop, third-party IDEs, and any MCP-compatible client.

Every diagnostic session the agent processes is also structured signal — a record of what was asked, what the tools returned, and what the resolution was. At scale, that accumulation becomes an organizational asset: failure patterns surface, recurring issues are recognized, institutional knowledge survives engineer turnover. The long-term product is not just a query tool — it is a diagnostic intelligence layer for IRIS environments that grows more valuable the more it is used.

Immediate next step: Publish as open source. An open-source release gives every Ensemble customer frictionless access, builds community around the tool, and establishes credibility with InterSystems and the SI partner ecosystem before any commercial motion begins. From there: a licensable offering for organizations that want managed packaging, support SLAs, and PHI-compliant deployment — distributed through the partner channel that already deploys and manages Ensemble productions for health systems, payers, and government agencies worldwide.